

Fixed-Weight Optimization of Qwen3-VL-2B Inference on PPU for the AICAS 2026 Challenge

AICAS PPU Optimization Team
Technical Report
China

Abstract—This report summarizes a fixed-weight inference optimization system for the AICAS 2026 PPU track. The target model is Qwen3-VL-2B-Instruct and the submission is constrained to preserve the pretrained checkpoint weights while optimizing the execution path. The final submission reached 2.3803 score, 0.7178 accuracy, 19.78 ms TTFT, and 369.80 tokens/s throughput. The final system combines profiling-driven visual input control, direct prompt KV reuse, load-time auxiliary W8 representations for selected decode projections, a Mega Kernel decode path, reduced grid synchronization, shared-memory staging for the MLP down projection, and dead-argument trimming in the fused decode call graph. The design focuses on stable single-path inference rather than metric-specific branching.

Index Terms—vision-language model, PPU, inference optimization, quantized inference, kernel fusion, Qwen3-VL, TTFT, throughput

I. INTRODUCTION

Large vision-language models (VLMs) are increasingly evaluated not only by answer quality, but also by latency and generation throughput. The AICAS 2026 PPU track emphasized this trade-off: a solution had to preserve answer accuracy while reducing time-to-first-token (TTFT) and increasing decode throughput on a constrained accelerator runtime. The final system described in this report targets Qwen3-VL-2B-Instruct and keeps the pretrained model checkpoint unchanged. All improvements are implemented in preprocessing, runtime dispatch, auxiliary inference-only weight layouts, and custom kernels. The work builds on the Transformer inference structure [1], the Qwen-VL model family [2], and established quantized-inference practice [3].

The optimization process was deliberately empirical. Many candidate changes produced local speedups but damaged the hidden online accuracy. Others improved only one latency metric while regressing throughput. The successful path was to preserve the semantic behavior of the model, make visual and text execution more predictable, and then remove overhead from the decode critical path. The final online result was:

- composite score: 2.3803,
- accuracy: 0.7178,
- TTFT: 19.78 ms,
- throughput: 369.80 tokens/s.

II. TASK AND CONSTRAINTS

A. Evaluation Metrics

The official benchmark reports four metrics: a composite score, answer accuracy, TTFT, and decode throughput. The

composite score is treated as the leaderboard objective, while accuracy is a hard practical constraint because many speed-only changes can reduce the final ranking. Local validation used the same benchmark interface on 150 samples, and online metrics were recorded after official submission.

B. Fixed-Weight Requirement

The submitted model does not fine-tune or edit the pretrained checkpoint. The implementation is allowed to construct auxiliary inference layouts at load time, such as W8 integer copies and scale arrays for selected projections, but the source checkpoint remains fixed. This distinction is important: the system uses quantized inference representations as an execution optimization, not as a new trained model.

C. Single-Path Policy

During development we explicitly rejected a strategy that uses low visual tokens for performance measurement and high visual tokens for answer generation. Although such a split can improve reported speed and accuracy independently, it makes the measured path no longer represent the answer path. The final report therefore focuses on stable optimizations that preserve a single operational policy and do not rely on metric-specific branching.

III. FINAL SYSTEM OVERVIEW

The submitted system contains a HuggingFace-compatible model wrapper, visual preprocessing, text prefill acceleration, and a custom decode extension. The final optimization targets three broad bottleneck groups:

- visual and multimodal prefill cost, which dominates TTFT;
- decode projection cost, especially Q projection and MLP gate/up/down paths;
- synchronization and launch overhead inside the fused decode loop.

The final scheme follows two complementary directions. First, it reduces redundant work before the first token. Second, it reduces per-token work inside the decode loop without changing the pretrained checkpoint.

IV. OPTIMIZATION METHOD

A. Visual and Prefill Path

The visual path was optimized conservatively. The runtime keeps a bounded image pixel budget, uses direct resize poli-

cies, caches vision positional information, and avoids unnecessary embedding work around the image block. The text prefill path uses packed QKV and packed gate/up big-ops where possible. For the final line of submissions, prompt KV reuse was used to avoid repeating prefill work when transferring from TTFT measurement to decode throughput.

The final version does not rely on aggressive visual-token deletion. This keeps OCR, small text, and fine-grained visual details available to the language model while still bounding the prefill workload through the image budget and cache-based optimizations.

B. Decode Weight Layouts

The decode path constructs auxiliary W8 representations for the Q projection and MLP projections. The key engineering choice is that the fused decode kernel reads INT8 weights and per-row scales directly for the hot projections, instead of performing Python-level eager decode or repeated conversion. The active W8 paths are branchless in the inner matvec loops:

$$y_m = s_m \sum_k q_{m,k} x_k, \quad (1)$$

where $q_{m,k}$ is an INT8 value and s_m is the row scale. This preserves the checkpoint while reducing memory bandwidth and improving cache locality for selected projections.

The final version uses W8 for the selected hot projections because it provides a stable trade-off between memory bandwidth, numeric robustness, and PPU memory alignment requirements. The packed layout is consumed directly by the decode kernel, avoiding Python-level unpacking or eager fallback execution.

C. Mega Kernel Decode Design

The core of the submitted runtime is a Mega Kernel for autoregressive decode. Instead of launching separate kernels for layer normalization, projections, attention, MLP, and logits, the implementation launches one persistent decode kernel per generation segment. The layer loop is executed on the device side, and intermediate buffers for the hidden state, residual stream, Q/K/V tensors, attention output, MLP intermediate, and logits remain in device memory across phase boundaries. The host mainly supplies the initial token, cache position, barrier state, and output buffers; it does not orchestrate every transformer sub-operator.

The Mega Kernel fuses the per-token transformer pipeline as follows:

$$\text{Embed} \rightarrow \left[\begin{array}{c} \text{RMSNorm+QKV} \\ \rightarrow \text{QKNorm+RoPE+KVWrite} \\ \rightarrow \text{Attention} \\ \rightarrow \text{OProj+PostNorm+MLP} \end{array} \right]^L \quad (2)$$

$\rightarrow \text{FinalNorm} \rightarrow \text{LMHead}.$

This design reduces host launch overhead and avoids repeated device-host handoffs between sub-operators. Within one token step, each layer is divided into coarse phases:

input RMSNorm and QKV projection, Q/K normalization with RoPE and KV-cache write, attention, and OProj+PostNorm+MLP. Global phase boundaries are coordinated by an atomic grid barrier rather than cooperative grid synchronization, which makes the kernel compatible with the available PPU runtime and keeps synchronization explicit.

The Mega Kernel also consumes the active W8 projection layouts directly inside the fused matvec loops. Q projection and MLP gate/up/down projections are read as INT8 values with per-row scales, while K/V projection and numerically sensitive state remain in the established representation. This hybrid layout keeps the hot decode projections bandwidth-efficient without changing the pretrained checkpoint.

Finally, the submitted kernel avoids an unnecessary bootstrap full-grid barrier. The host launch path already clears the barrier counter and generation flag before each decode launch, so the first useful synchronization can advance the generation directly. This small synchronization cut matters because the same decode phases repeat at every generated token.

D. MLP Down-Projection Data Flow

The OProj+PostNorm+MLP phase is one of the main decode hotspots. The MLP intermediate has width 6144. The final layout stages this intermediate in an unpadding dynamic shared-memory page before the down projection, while keeping the smaller static shared vectors padded where that helps QKV, O projection, and normalization reads. This keeps the down-projection consumer path compact without increasing dynamic shared-memory footprint.

E. Dead-Argument and Branch Trimming

The final implementation removes stale prefetch and fallback arguments from the fused decode call graph. Unused pointers such as next-layer Q/K/V weights and unused gate/up/down arguments were removed from device function signatures. Although these values were never read, they still increased call boundary pressure and could affect register allocation and code generation.

The final version removes dead arguments from QKV, QK/RoPE, attention, and OProj+MLP boundaries. It also specializes the Phase-4 MLP to the active W8 gate/up/down path and drops the dead FP16 fallback branches. This does not change the math of the active submission path, but it simplifies the compiled kernel used by the final submission.

V. FINAL SUBMISSION

The final official result is summarized in Table I. These are the official online metrics for the submitted fixed-weight system.

VI. DISCUSSION

The final scheme favors stable execution over aggressive compression. W8 inference layouts match the PPU memory model and allow branchless inner loops. Mega Kernel decode fusion reduces host overhead and makes synchronization explicit. Once the kernel is fused and memory-bound, dead pointers and unused branches can still perturb code generation, so

TABLE I
FINAL OFFICIAL SUBMISSION METRICS

Metric	Value
Composite score	2.3803
Accuracy	0.7178
TTFT	19.78 ms
Throughput	369.80 tokens/s

trimming the active call graph is part of the final performance strategy.

The final submission also avoids metric-specific path splits. The same operational policy is used for answer quality and performance-oriented execution, which makes the reported TTFT and throughput representative of the submitted inference system.

VII. CONCLUSION

This report presented a fixed-weight optimization pipeline for Qwen3-VL-2B inference on PPU. The final submission achieved 2.3803 official score, 0.7178 accuracy, 19.78 ms TTFT, and 369.80 tokens/s throughput. The final design combines conservative visual preprocessing, prompt KV reuse, W8 auxiliary decode layouts, a Mega Kernel decode path, reduced grid synchronization, MLP down-projection shared-memory staging, and decode call-graph trimming. The system achieves these metrics without modifying the pretrained checkpoint.

REFERENCES

- [1] A. Vaswani et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems*, 2017.
- [2] J. Bai et al., “Qwen-VL: A versatile vision-language model for understanding, localization, text reading, and beyond,” arXiv:2308.12966, 2023.
- [3] A. Gholami et al., “A survey of quantization methods for efficient neural network inference,” arXiv:2103.13630, 2021.